

Практика 9. Применение аппликативных функторов

Андрей Стоян, Илья Колегов, Дмитрий Халанский

MSE ITMO

осень 2024

В предыдущих сериях

- ▶ `Functor`, `Applicative`
- + `Traversable`, `Alternative`
- + Аппликативные парсеры

1. Traversable
2. Парсер-комбинаторы
3. Parsec (*)

Содержание

1. Traversable

2. Парсер-комбинаторы

3. Parsec (*)

Traversable

```
class (Functor t, Foldable t) => Traversable t where
  sequenceA :: Applicative f => t (f a) -> f (t a)
  traverse  :: Applicative f => (a -> f b) -> t a -> f (t b)
```

- ▶ **Functor** — обработать значения в контейнере
- ▶ **Foldable** — аккумулялировать значения в контейнере
- ▶ **Traversable** — обработать значения и аккумулялировать структуру

todo Реализуйте **Functor** через **Traversable**

todo Реализуйте **Foldable** через **Traversable**

```
⚡ fmapDefault :: Traversable t => (a -> b) -> t a -> t b
fmapDefault f = getIdentity . traverse (Identity . f)

⚡ foldMapDefault :: (Traversable t, Monoid m) => (a -> m) -> t a -> m
foldMapDefault f = getConst . traverse (Const . f)
```

Применение Traversable

todo `GHCI> sequenceA [("a", 1), ("b", 2)]`

todo Достаньте из реестра всех пользователей по списку имён

todo Изобретите цикл `for` и распечатайте в “консоль” каждое число из списка по два раза



```
("ab", [1, 2])
```



```
GHCI> traverse (usersDatabase !?) ["fred", "bob", "alice"]
Just [User {..}, User {..}, User {..}]
```



```
for = flip traverse
```

```
GHCI> for [1, 2, 3] $ \x -> print x *> print x
```

```
1 1 2 2 3 3
```

```
[(), (), ()]
```

Содержание

1. Traversable

2. Парсер-комбинаторы

3. Parsec (*)

Парсер

△ Преобразует менее структурированные данных в более структурированные

e.g. Текст на языке программирования → дерево абстрактного синтаксиса

e.g. Байты из сети → структура данных запроса пользователя

e.g. Опции командной строки → объект конфигурации программы

e.g. Нажатия на кнопки в игре → игровые действия

Как пишут парсеры (☺)

▶ Руками

e.g. Рекурсивный спуск, shift reduce. . .

- + Очень быстрые, кастомизируемые, хорошие сообщения об ошибках
- Долго, сложно, противно писать и поддерживать
- ▶ Хороший выбор для промышленных парсеров сложных языков

▶ Генерируют по описанию на специальном языке

e.g. Antlr, yacc, bison, tree-sitter, protobuf. . .

- + Просто описать грамматику в приятном синтаксисе
- Для сложных языков либо медленные, либо нужна уродливая специальная грамматика
- Должна быть реализация для мета-языка и интеграция с его системой сборки
- Может быть проблемой интеграция в большие проекты
- Плохо годятся для тонкой настройки

▶ Парсер-комбинаторы — язык для конструирования парсеров

e.g. Parsec, [ts-parsec](#), [compynator](#), [jparsec](#). . .

- + Полностью реализуются средствами языка, просто писать и парсеры и комбинаторы
- + По скорости могут быть сопоставимы со сгенерированными парсерами
- Производительность и кастомизируемость ограничены, нужны навыки для использования

Парсеры своими руками

```
newtype Parser a = Parser { runParser :: String -> Maybe (String, a) }
```

todo Реализуйте парсер цифр `Parser Char`

todo На его основе реализуйте парсер `Parser Int`

todo Реализуйте парсер номера домашки как пару цифр `"09" -> (0,9)`

todo Реализуйте парсер пар цифр `"(0,9)" -> (0,9)`

```
⚡ digitC = Parser \s -> case s of
    c:cs | Data.Char.isDigit c -> Just (c, cs)
    _ -> Nothing
```

```
⚡ digit = Data.Char.digitToInt <$> digitC
```

```
⚡ hwId = (,) <$> digit <*> digit
```

```
⚡ pairD = between (char '(') (char ')') $
    (,) <$> digit <*> char ',' <*> digit
```

Альтернатива

```
class Applicative f => Alternative f where
  empty :: f a
  (<|>) :: f a -> f a -> f a
```

todo Реализуйте комбинатор `some`

todo Реализуйте парсер чисел

```
⚡ some, many :: Alternative f => f a -> f [a]
many p = (:) <$> p <*> many p <|> pure []
some p = (:) <$> p <*> many p

⚡ number :: Parser Int
number = read <$> some digit
```

Содержание

1. Traversable

2. Парсер-комбинаторы

3. Parsec (*)

Поток токенов

```
class Stream s m t | s -> t
```

- ▶ Способ получать токены типа t из оставшегося потока типа s
- ▶ Оперирует в аппликативном функторе m (не очень важно для нас)
- ▶ Обладает единственной операцией `uncons :: s -> m (Maybe (t, s))`
- ▶ s уникально определяет t (для потока `[Token]` это `Token`)¹

e.g. `instance Stream String Identity Char` — способ получать символы из строки

¹FunctionalDependencies.

Парсер

Тип `ParsecT s u m a`:

- ▶ Принимает на вход источник типа `s`
- ▶ Поддерживает внутри пользовательское состоянием типа `u`
- ▶ Оперирует в аппликативном функторе `m`
- ▶ Парсит экземпляр `a`

e.g. `ParsecT String () Identity Int` — парсит `Int` из `String` без доп. состояния и эффекта

- ▶ `type Parsec s u a = ParsecT s u Identity a`
- ▶ Тестирование в GHCi:

```
parseTest :: (Stream s Identity t, Show a) => Parsec s () a -> s -> IO ()
```

Далее в слайдах используем `type Parser a = Parsec String () a`.

Примеры комбинаторов Parsec²

```

eof :: (Stream s m t, Show t) => ParsecT s u m ()
spaces :: Stream s m Char => ParsecT s u m ()
alphaNum :: Stream s m Char => ParsecT s u m Char
char :: Stream s m Char => Char -> ParsecT s u m Char
between
  :: Stream s m t
  => ParsecT s u m open
  -> ParsecT s u m close
  -> ParsecT s u m a
  -> ParsecT s u m a
...

```

²Parsec documentation.

Семантика парсера как представителя Functor

- Сделать дополнительные действия с результатом парсинга

e.g. `isDigit :: Char -> Bool`
`satisfy :: (Char -> Bool) -> Parser Char`
`digit :: Parser Int`
`digit = digitToInt <$> satisfy isDigit`
-- ^ Парсит цифру и возвращает символ
-- ^ Превращает символ в число

Семантика парсера как представителя `Applicative`

- ▶ `pure x :: Parser A` — не потребляет строку, возвращает значение
- ▶ `(<*>)` — сначала парсит левым парсером, потом правым

e.g. `digit' :: Parser Int`

```
digit' = pure digitToInt <*> satisfy isDigit
```

e.g. Парсит две цифры подряд `"34" -> (3, 4)`

```
digit2 :: Parser (Int, Int)
```

```
digit2 = (,) <$> digit <*> digit
```

e.g. Парсит цифры через запятую `"(2,3)" -> (2, 3)`

```
pair :: Parser (Int, Int)
```

```
pair = between (char '(') (char ')')
```

```
  $    (,)      -- :: Int -> Int -> (Int, Int)
```

```
  <$> digit    -- :: Parser (Int -> (Int, Int))
```

```
  <*> char ',' -- :: Parser (Int -> (Int, Int))
```

```
  <*> digit    -- :: Parser (Int, Int)
```

Семантика парсера как представителя *Alternative*

Семантика ($\langle | \rangle$) (пусть $p = p_1 \langle | \rangle p_2$)

- ▶ Запускает левый парсер p_1
- $\Rightarrow$ Если p_1 падает **на первом же токене**, запускает p_2 на том же входе
- $\Rightarrow$ Если p_1 падает **не на первом токене**, весь парсер p завершается неудачей (p_2 не запускается)
- ▶ Заставляет писать однопроходные парсеры
- ▶ Семантика с лекции позволяет легко написать экспоненциальный парсер

Семантика *try*:

$\Delta \text{ try} :: \text{ParsecT } s \text{ u m a} \rightarrow \text{ParsecT } s \text{ u m a}$

- ▶ Парсер $\text{try } p_1$, упав не на первом токене, притворяется, что упал на первом
- ▶ Запись $\text{try } p_1 \langle | \rangle p_2$ в семантике Parsec ведёт себя так же, как и $p_1 \langle | \rangle p_2$ на лекции

Грабли при использовании Alternative

- Будем парсить параметры функции, передаваемые через пробел:

```
param :: Parser Term
param = between (char '(') (char ')') term <|> var
params :: Parser [Term]
params = many $ spaces *> param
```

- На строчке "x y !" params говорит: Unexpected symbol '!'. Вместо того, чтобы распарсить префикс и вернуть хвост ("!"). В чем беда?
- Вспомним как реализуется many:

```
many :: Parser a -> Parser [a]
many p = ((:) <$> p <*> many p) <|> pure []
```

- Альтернатива! Наш парсер съедает пробел перед '!', а дальше встречает неожиданный символ и весь завершается с ошибкой, потому что этот символ уже не первый
- Как исправить? Парсить сначала параметр, а потом пробел.³

```
params' :: Parser [Term]
params' = spaces *> many (param <*> spaces)
```

³Лучше всего избавиться от пробелов на фазе лексического анализа.